DisasterShield AI: Team Explainer

What it is, what each screen does, who builds what, and how the pieces join

TinyFish · Kritical Hacks '26 · 17 Oct 2026 · short version

How to read this

Sections 1 to 4 explain the product in plain words. **Section 5 says who builds what. Section 6 says how the parts join.** Details marked **proposal** are our own design choices, not hackathon rules, so the team can change them together.

1. DisasterShield AI in one minute

It warns people about a disaster, helps them reach safety even with no internet, and gives district authorities verified information to direct rescue.

Part	What it is	Built with
Hazard Radar	Background program that reads official sites (IMD, CWC, NDMA, state portals) through TinyFish and turns bulletins into alerts with a source link.	Python · TinyFish Search / Fetch (Agent for shelters)
Citizen app	Phone web app (PWA). Alerts, shelter map, SOS button. Saves everything in advance so it works with no signal.	React · Tailwind · Leaflet · Service Worker · IndexedDB
Authority dashboard	Laptop page for the district control room: heatmap, SOS queue, photo reports, shelters, officer actions.	React · Leaflet · Node.js · PostgreSQL + PostGIS

	Citizen	Authority officer
Who / device	Any resident, on a phone	District Disaster Management Authority staff, on a laptop
Account	Quick set-up: name, phone, home district (OTP mocked)	Created by an admin, tied to one district (seeded in demo). JWT role: authority
Cost	Always free	Paid by the district licence

Five rules the product follows

1	**Prepare first, connect second.** Alerts, shelters and map tiles are saved on the phone while online.
2	**Nothing disappears silently.** Every SOS or report shows a status (Queued, Sent, Received...).
3	**AI assists, humans decide.** The model gives a score; an officer approves or rejects.
4	**Every alert shows its source and time**, with a link to the official page.
5	**Privacy by role.** Citizens never see other people's SOS or phone numbers; only that district's officers do.

2. System architecture

Read top to bottom: data starts on the live web, passes through TinyFish and our code, is stored, then reaches the two apps. Owners (M1 to M4) are explained in section 5.

1 · Live web: official sources
IMD bulletins · CWC flood data · NDMA advisories · State SDMA portals · News sites

▼ public pages

2 · TinyFish: our access to the live web
Search (CORE, free): newest alerts per district. **Fetch** (CORE, free): clean text of official pages.
Agent (STRETCH, credits): goal-driven task such as collecting shelter lists. **Browser** (STRETCH, credits): JavaScript-heavy pages.

▼ page text

3a · Radar worker (Python) · M3
extracts hazard, severity, district, validity · removes duplicates · keeps raw snapshots
official source = auto-publish; news source = pending for an officer

3b · AI photo check (Python) · M3
pretrained flood / damage model · returns label + confidence · assists the officer, never dispatches alone

▼ alerts + source link · label + score

4 · Node.js / Express API + PostgreSQL / PostGIS · M2
JWT roles (citizen, authority) · alerts · shelters · SOS · reports · heatmap · idempotent /api/sync · mock SMS fallback
Tables: users · alerts · shelters · sos_events · reports · radar_snapshots (key fields: `client_id` unique on SOS and reports; alerts hold hazard, severity, district, valid_until, source_url, fetched_at, origin, status)

▼ JSON over HTTPS · alerts reach phones by in-app feed, Web Push (needs internet) or SMS fallback (mocked)

5 · Citizen app (phone PWA) · M1 + M4
Tabs: Home · Map · SOS · Report · Profile
Service Worker cache: app, last alerts, shelters, map tiles
IndexedDB queue: SOS and reports, sent via /api/sync

6 · Authority dashboard (laptop) · M1 + M4
Pages: Overview · Alerts · SOS · Reports · Shelters
Command map: heatmap, SOS pins, shelters, alert zones
Needs internet (not offline-first)

3. Three stories

A · An official bulletin becomes an alert on a phone

1	The Radar runs on a timer for each monitored district (proposal: every 15 min). TinyFish **Search** finds the newest cyclone, flood, rain or quake pages; **Fetch** returns clean text, saved as a timestamped snapshot.
2	The extractor builds an alert: hazard, severity, district, valid-until, source link, fetched-at. Duplicates (same hazard + district + source within a few hours) update the old alert.
3	**Publish rule (proposal):** official sites (IMD, CWC, NDMA, state portals) publish automatically; news sites become Pending until an officer approves. Officers can always edit or withdraw.
4	Citizens in the alert area get it in the app (high severity also by push, stretch). The phone keeps a copy, so it still shows with an "updated at" time if the signal drops.

B · An SOS

1	Citizen holds SOS for 2 s; a 5 s countdown allows cancelling. The phone saves GPS, time, phone number and a unique `clientId` **on the phone first**.
2	Online: sent at once to POST `/api/sos/trigger`. Offline: stays queued in IndexedDB and an SMS fallback is attempted (mocked).
3	When any signal returns, the queue goes to POST `/api/sync`; the server uses clientId so nothing is counted twice.
4	Officers see a red SOS in the queue and on the map, and press Acknowledge, Dispatch, Resolve. The citizen sees each status when next online.

C · A photo report

1	Citizen picks a type (e.g. Flooding), takes a photo (shrunk on the phone) and confirms location. It is sent now or queued.
2	The API sends the photo to the AI check, which returns a label and confidence. **Rule (proposal):** 70% or more **and** label matches the chosen type = AI-verified and counted on the heatmap; otherwise Needs review (not on the heatmap until approved).
3	The officer looks at the photo and presses Approve or Reject; the citizen sees the result in My reports.

Be honest in the demo and README

The photo model is pretrained (not trained by us) and the 70% threshold is a starting guess. SMS gateway and OTP are mocked. Say so.

4. Screens and buttons

Citizen app (phone) · five tabs: Home, Map, SOS, Report, Profile

Set-up (1 min): add the app to the Home Screen, pick language (stretch), home district, name and phone (mock OTP), allow location and notifications. The app then caches the district's alerts, shelters and map tiles and shows "Ready for offline use". **Guest mode (proposal):** without set-up a visitor can read alerts and see shelters but cannot send SOS or reports, because an SOS needs a phone number for call-back.

Tab	Element → what it does	Offline?
Home	**Status strip** shows Online / Offline + updated time (tap to sync). **Alert cards**: severity colour (High red, Medium orange, Low yellow), hazard, area, valid-until, source; sorted by severity then newest. **Filter chips** by hazard. **Detail:** What to do checklist (static text), Show shelters (jumps to Map), View source (opens official page).	Yes (View source: No)
Map	Blue GPS dot, layer toggles (Shelters, Danger zones), shelter pins coloured by status, red danger zones (areas of active alerts). **Nearest open shelter** centres on the closest Open or Filling shelter with a dashed route line. Shelter card: Show route, Call, Open in Maps.	Yes (turn-by-turn: No; Call: partly)
SOS	**Hold 2 s** starts a 5 s countdown with Cancel. **Status card:** Queued, Sent, Received, Acknowledged, Dispatched, Resolved. **Add details** (people, injured, note) updates the same SOS. **I'm safe, cancel SOS. Call 112.**	Yes, queued (status: last known)
Report	Type chips (Flooding, Building damage, Road blocked, Landslide, Fire, Other), camera or gallery (photo shrunk to about 300 KB), draggable location pin, note, Submit. **My reports** shows Queued, Under review, Verified or Rejected.	Yes, queued
Profile	Home district (decides alerts and the offline area), phone number (only that district's officers see it), notifications toggle, Download offline area, Sync now ("2 items waiting"), Log out.	Mostly; district change, download and sync need internet

Cannot do offline: receive NEW alerts (SMS fallback is a later step), turn-by-turn directions, and the whole authority dashboard. Map tiles exist only for the zoom levels that were saved.

Authority dashboard (laptop) · officers see only their own district

Left menu (Overview, Alerts, SOS queue, Reports, Shelters); top bar with district, Online indicator, bell for new SOS and reports. **Flow:** alert or citizen SOS / photo → appears in its queue → officer verifies → officer acts → citizen sees the new status.

Page	Element → what it does
Overview	KPI tiles (Active alerts, Open SOS, Reports to review, Shelters open / capacity) open the page filtered. Layer toggles: Heatmap, SOS pins, Shelters, Alert zones. Time filter 1 h / 6 h / 24 h. Click a heatmap cell to list its incidents; click a pin to open its card.
Alerts	Table: hazard, severity, area, source, fetched time, status (Published, Pending, Withdrawn, Expired). **Approve & publish** (Pending), **Edit**, **Withdraw** (records a reason), **View source**. **New manual alert** (STRETCH): hazard, severity, area, message, valid-until.
SOS queue	Cards, oldest unacknowledged first: time, phone, people, injured, note, status. 3 or more SOS within about 500 m get a **Cluster** badge (proposal). **Acknowledge**, **Dispatch** (asks for team name), **Resolve** (leaves the heatmap), **Call citizen**, **Show on map**, **Mark duplicate**.
Reports	Tabs: Needs review, AI-verified, Rejected. Card: photo, AI label with confidence bar, mini-map, note. **Approve** = Verified and counted on the heatmap. **Reject** with reason (unrelated, duplicate, unclear). Officers can override an AI-verified item: the human decision wins.
Shelters	Table + map with capacity and occupancy. **Update occupancy** (+/-): Open below 80%, Filling from 80%, Full at 100%. Open/Closed toggle (closed = grey to citizens). Add / Edit and Import from TinyFish Agent are STRETCH (demo can seed with SQL).

5. Rules, visibility and statuses

Who sees what	Guest	Citizen	Officer (own district)
Published alerts, source links, shelter map, danger zones	Yes	Yes	Yes (+ Pending, Withdrawn)
Own SOS and reports, with status	No	Yes	Yes
Other people's SOS, phone numbers; all reports with AI score; KPI counts; heatmap	No	No	Yes
Edit shelters and occupancy	No	No	Yes

The heatmap is officers-only (proposal) because it shows where individuals ask for help: a privacy risk that could also cause panic. Citizens see only the coarse danger zones from official alerts.

Event	Who is notified	How
Official alert published	Citizens in the area (home district, or GPS when online); officers	In-app feed; high severity also Web Push (stretch)
Pending alert (news source)	Officers	Alerts page + bell
New SOS / new report needing review	Officers of that district	Queue + bell + map pin
SOS status change, report decision	The citizen who sent it	SOS tab, My reports (push is stretch)

Web Push needs internet; on iPhone only when the app is on the Home Screen. Citizens create SOS and reports; the Radar creates alerts; the AI check creates labels and scores; officers make every decision.

Status chains

Object	Chain
SOS	Queued (on phone) → New → Acknowledged → Dispatched → Resolved | Cancelled | Duplicate
Report	Queued → Under review (AI) → AI-verified or Needs review → Verified | Rejected
Alert	Pending (non-official) → Published → Expired | Withdrawn (official alerts start at Published)
Shelter	Open → Filling (80%) → Full | Closed. Pins: green Open, amber Filling, red Full, grey Closed. Data older than 6 h shows a warning (proposal).

Heatmap: the map is split into a grid; GET /api/heatmap returns a count per cell for verified or AI-verified reports and unresolved SOS in the chosen time window, coloured yellow (few) to red (many). It shows what has been reported and verified, not a prediction. Use: decide where to send rescue first.

6. Who builds what

Each member owns a lane and builds it against the agreed contract, using mock data so nobody waits. Lanes meet at five **seams** (S1 to S5, explained in section 7). Tags: **MUST**, **STRETCH**, **MOCKED** follow the build scope.

Folder / file	Owner	Others
`web/src/pages/, web/src/lib/ (API client, offline queue, service worker)`	M1	PR only
`web/src/components/ (SOS button, alert card, shelter list, banner, dashboard widgets)` (proposal)	M4 builds, M1 reviews	M1 imports
`api/ (routes, auth, migrations); .env.example`	M2	PR only; add own variables by PR
`api/seed/, api/tests/ (seed files, Postman, smoke tests, SMS log view)`	M4 writes, M2 merges	M2 loads seed
`radar/, vision/`	M3	PR only
`radar/tests/, radar/sources.md, docs/, README top section, demo assets`	M4	M3 runs the tests

Member 1 (M1)

Frontend and UI lead · React, Tailwind, Leaflet, Service Worker

What to build	Done when
App shell (MUST). React + Tailwind PWA with routing, a `USE_MOCK` switch in `web/src/lib/api.js`, built first on `mockData.json` (3 alerts, 4 shelters, map points).	Runs on phone and laptop width; flipping USE_MOCK changes the data source and nothing else.
Citizen screens (MUST). Home, Map, SOS, Report, Profile exactly as in section 4 (SOS, Home, Map first on Sat). Generate a `clientId` (crypto.randomUUID) when the SOS hold starts.	Mock alerts and shelters render; an SOS saves on the phone first and shows a status.
Offline layer (MUST, seam S4). Service Worker caches app shell, last API replies and home-district map tiles. IndexedDB queue holds SOS and reports. On the `online` event, app open and Sync now, send the queue to `POST /api/sync` and update each item's status from the reply.	Airplane mode: app still opens; SOS queues; reconnect empties the queue with no server duplicate.
Authority dashboard (MUST, seam S5). Login, KPI tiles, Overview map with heatmap and SOS pins, SOS queue buttons, Reports Approve / Reject, Alerts list + Withdraw. **STRETCH:** Shelters editing, manual alert, approve pending.	An officer takes one SOS from New to Resolved and one report to Verified.

Do not touch: backend routes, SQL, schema, radar or vision code. Use mock JSON until M2's endpoints are live.

Member 2 (M2)

Backend and database architect · Node.js, Express, PostgreSQL, PostGIS

What to build	Done when
Repo and contract (MUST). Public repo, README skeleton, branch rules, license; write the API contract with example JSON; review and merge every PR.	All four members can branch and push; contract agreed.
Schema, seed, auth (MUST). PostGIS tables (section 2), seed loader for M4's files, signup / login, JWT roles; authority sees only their district, a citizen only their own SOS.	One command builds and seeds; a citizen token cannot read another person's SOS.
Alerts and shelters (MUST, seam S1). GET /api/alerts, GET /api/shelters. Alert ingest via POST /api/alerts (proposal: header `X-Radar-Key`); merge duplicates; PATCH for approve, edit, withdraw.	Real rows in the contract shape by Sat evening; same alert posted twice = one row.
SOS and sync (MUST, seam S4). POST /api/sos/trigger idempotent on client_id; GET /api/sos and PATCH (Acknowledge, Dispatch, Resolve, Duplicate, Cancel); POST /api/sync processes each queued item separately and replies per item. Mock SMS log.	Same clientId twice = one row; one failed item never blocks the rest.
Reports, verify, heatmap (MUST, seam S3). POST /api/reports calls the verify URL and applies the 70% + matching-type rule; GET / PATCH for Approve and Reject; heatmap grid for 1 / 6 / 24 h. **Mock first:** hardcode `{"confidence": 0.85, "label": "flood"}`.	Report gets status and score; swapping the mock for M3's service needs only an env change.

Do not touch: CSS or React; radar or model code. Consume M3's work only through the alerts table and the verify endpoint.

Member 3 (M3)

AI and GIS specialist · Python, TinyFish, pretrained vision model

What to build	Done when
TinyFish and radar loop (MUST). Account and key in `TINYFISH_API_KEY`; `tinyfish_client.py` wraps Search + Fetch (timeouts, retries). `run.py`: per district, Search → Fetch → save snapshot → extract → de-duplicate. Include a replay mode that runs from saved snapshots.	One command logs a live Fetch and writes alerts; replay works with the network off.
Extractor and publish rule (MUST). `extract.py`: text → {hazard, severity, district, validUntil, sourceUrl} with simple severity rules and district matching; unsure = Pending. Official domains (imd.gov.in, cwc.gov.in, ndma.gov.in, state SDMA) = Published; news = Pending.	M4's sample bulletins give the expected output.
Hand-off to M2 (seam S1). Sat: write `radar/output/alerts.json`. Sun: post each alert to M2's POST /api/alerts with the radar key.	Alert shows on M1's Home tab with source link and fetched time.
Vision service (MUST, seam S3). Pretrained flood / damage model behind `POST /api/ai/verify-image` returning {label, confidence}. Add a mapping from model labels to the report types (Flooding, Building damage, Road blocked, Landslide, Fire, Other) so "label matches chosen type" works.	10 test images give sensible labels; weak cases listed honestly in the README.
Agent shelters (STRETCH). Run the TinyFish Agent once for one district and cache to `radar/output/shelter_candidates.json`. Never run it live in the demo.	File exists; M2 or an officer imports it.

Do not touch: the main web server or the frontend. Expose a worker and one HTTP endpoint that M2 calls.

Member 4 (M4)	Product, delivery and technical support · README, video, QA, React, API tests, data
What to build	**Done when**
UI components for M1. From mockData.json build SOS button, alert card, shelter list, offline banner, dashboard layout, photo upload box and AI confidence badge in `web/src/components/`. Feature branches only.	M1 imports each component without editing it.
Seed data and tests for M2. Seed JSON/CSV, mock-mode docs, Postman collection and smoke script for every endpoint (including same-clientId-twice on SOS and sync), mock SMS log view.	Smoke script passes on every endpoint.
Sources and tests for M3. Three official sources per hazard per district (`radar/sources.md`), sample bulletins with expected extractor output, and a table of the 10 test images with label and confidence.	Extractor tests pass; image table ready for the README limits section.
QA and delivery. Run the end-to-end script (section 7) every evening before the 9 PM handshake and log bugs with the owner's name. Own README top section, demo video, GIF, social post, Unstop submission, last-slide links, fresh clone test.	No open MUST bugs by Mon 9 PM; repo, video and post open in a private window.

Do not touch: the main branch directly. Every change goes through a feature branch and a PR reviewed by M2.

Hand-off schedule: who gives what to whom

#	From → To	What exactly	When	Receiver checks
1	M2 → all	API contract with example JSON (`docs/api-contract.md`)	Fri night	Fields match what each side needs
2	M1 → M2, M4	mockData.json in the contract shape	Sat morning	Shape equals the contract
3	M4 → M2	Seed JSON/CSV, first Postman collection	Sat	Loads with one command
4	M3 → M2	alerts.json from the radar	Sat evening	Every row loads into alerts
5	M2 → M1	Running API URL, CORS, seed logins	Sat evening	GET /api/alerts returns rows
6	M4 → M1, M3	Components; source list and extractor tests	Sat / Sun	Components render; tests run green
7	M3 → M2	Live alert posts and verify-image URL	Sun	Same alert twice = one row
8	M3 → M4	Radar log clip, sample response screenshot	Sun / Mon	Usable in README and video
9	All → M4	Fixes for logged bugs	Mon	QA script passes again

7. How the pieces join

1	**Contract first.** Nobody changes a field or endpoint without telling the team and updating the contract file.
2	**Mock until both sides are green.** Switch one seam at a time, only the two owners of that seam; if it fails, return to the mock and retry next morning.
3	**Small branches, daily push.** Everything is pushed by 11 PM even if it runs on mock data. M2 reviews PRs. Never commit keys; new variables go in `.env.example`.
4	**Label every mock** in the README and the video.

The five seams: switching from mock to real

Seam	Mock in use until	How to switch	Pass check	Who
S1 Radar → DB	M3 writes alerts.json; M2 loads it by script	M3 posts to POST /api/alerts with X-Radar-Key (proposal)	Same alert twice = one row, with source link and fetched time	M3 + M2
S2 API → Apps	M1 reads mockData.json	Set `VITE_USE_MOCK=false` and `VITE_API_URL`	Home tab shows the alert that is in the database	M1 + M2
S3 Report → Vision	M2 returns the fixed 0.85 / flood reply	M2 sets `VERIFY_URL` to M3's service (empty = mock)	10 test images logged; low score goes to Needs review	M2 + M3
S4 Queue → Sync	M1 tests queue locally; M2 tests with Postman	M1 points the reconnect handler at POST /api/sync	Airplane mode, SOS, reconnect: one row; resend: still one	M1 + M2
S5 Dashboard	M1 uses mock incidents	Dashboard calls /api/sos, /api/reports, /api/heatmap with an officer token	SOS cluster and verified photo show; citizen sees status change	M1 + M2, M4 verifies

Environment variables, one owner each: `TINYFISH_API_KEY` (M3), `DATABASE_URL`, `JWT_SECRET`, `RADAR_KEY`, `SMS_MODE=mock` (M2), `VERIFY_URL` (M2, points to M3), `VITE_API_URL`, `VITE_USE_MOCK` (M1).

Endpoint contract

The original contract has the first seven rows. M2 adds the rest before building (marked new).

Endpoint	Owner	Used by
`GET /api/alerts` · `GET /api/shelters`	M2	Home and Map; all dashboard pages
`POST /api/sos/trigger` · `POST /api/sync`	M2	SOS button; auto-sync on reconnect
`POST /api/reports` · `POST /api/ai/verify-image`	M2 / M3	Report tab; AI check
`GET /api/heatmap`	M2	Overview map
`POST /api/auth/signup` · `/login` (new)	M2	Set-up; officer login
`GET /api/sos?status=` · `PATCH /api/sos/:id` (new)	M2	SOS queue actions; citizen cancel
`GET /api/reports?status=` · `PATCH /api/reports/:id` (new)	M2	Approve, Reject
`POST /api/alerts` · `PATCH /api/alerts/:id` (new)	M2	Manual alert, Approve, Edit, Withdraw; radar ingest (proposal)
`POST /api/shelters` · `PATCH /api/shelters/:id` (new, stretch)	M2	Shelters page

Shared data shapes

Alert, SOS and verify follow the original contract. Fields marked **proposal** are extra; M2 confirms them on the kickoff call.

```
ALERT    { "id", "hazard": "cyclone", "severity": "high", "district", "validUntil", "sourceUrl",
           "fetchedAt", "origin": "radar", "status": "published" }        // origin, status: proposal
SOS      { "clientId": "uuid", "lat": 12.97, "lng": 80.22, "timestamp", "phone" }   // phone: proposal
SYNC     request { "sos": [ SOS... ], "reports": [ { "clientId", "type", "lat", "lng", "note", "photo" } ] }
         reply   { "results": [ { "clientId", "result": "accepted" | "duplicate" | "failed" } ] }  // proposal
VERIFY   in: image (+ chosen type)   out: { "label": "flood", "confidence": 0.85 }   // the mock reply
HEATMAP  { "windowHours": 6, "cellSizeMeters": 500, "cells": [ { "lat", "lng", "count" } ] }  // proposal
```

End-to-end test (M4 runs it every evening) and who fixes what

#	Action → expected result	If it fails, look first at
1	Run the radar once → live Fetch in the log; new alert row with source link and fetched time	M3 (S1), then M2
2	Open the citizen app online → alert on Home; shelters and map load; "Ready for offline use"	M1, then M2 (S2)
3	Airplane mode, reload → app opens; alert shows "updated at"; shelter map and route line work	M1 (service worker)
4	Hold SOS 2 s, let it finish → status Queued; SMS fallback (mocked) logged	M1, M2 (SMS log)
5	Airplane mode off → queue syncs; status Received; officer sees one new SOS. Resend same payload → no duplicate	M1 + M2 (S4)
6	Submit a flood photo → label and confidence; 70% or more and matching type = AI-verified	M2 + M3 (S3)
7	Officer dashboard → SOS cluster and verified photo on the heatmap; Approve, Reject, Resolve work; citizen sees new statuses	M1 + M2 (S5)
8	Fresh clone, README commands only → app runs	M2, M4

Fallbacks and "integrated"

Failsafe A: if M2's backend fails, M1 reads the static JSON files M3 generates (keeps the TinyFish points). **Failsafe B:** if the vision model breaks, keep M2's fake reply and say so in the video. M1 and M4 decide at the Mon 19 Oct 9 PM feature freeze whether an unfinished feature is deleted or mocked. **Integrated** means steps 1 to 8 pass on a fresh clone, every remaining mock is labelled, and all four members have commits on main.